

Documento B — Processo de Teste

Projeto: DietHub

Tecnologia: Flutter + Node.js + MySQL

Arquitetura: MVVM + API REST

Norma aplicada: ISO/IEC/IEEE 29119-2

1. Estratégia de Teste

O processo de testes será executado utilizando abordagem incremental e automatizada.

Serão utilizados:

- Testes de unidade
- Testes funcionais
- Testes de integração
- Testes baseados em cenário
- Simulação de APIs utilizando mocks
- Execução automatizada em Flutter

A estratégia prioriza os fluxos críticos:

- Cadastro
- Login
- Geração de plano alimentar
- Registro de refeições
- Navegação principal

2. Ambiente de Teste

Ferramentas

- Flutter SDK
- Dart SDK
- flutter_test
- integration_test
- Mock APIs
- Android Studio
- Visual Studio Code
- GitHub

Ambiente de Execução

- Android Emulator
- Dispositivo Android físico

Dependências

- Provider
- HTTP
- SharedPreferences

3. Critérios de Entrada

Os testes iniciarão após:

- Compilação correta do projeto
- Implementação das telas principais
- Configuração dos serviços
- Integração das APIs
- Documento A concluído
- Casos de teste definidos

4. Critérios de Saída

Os testes serão concluídos quando:

- Todos os casos forem executados

- Fluxos críticos forem aprovados
- Defeitos críticos forem corrigidos
- Evidências forem registradas
- Relatório final for produzido

5. Ordem de Execução

- Testar RegisterScreen
- Testar LoginScreen
- Testar HomeScreen
- Testar CreatePlanScreen
- Testar LogMealScreen
- Testar ScanPlanScreen
- Testar ApiService
- Testar AIService
- Testar AppState
- Testar fluxo Cadastro → Login
- Testar fluxo Login → Home
- Testar fluxo geração de plano
- Testar fluxo registro de refeições

6. Implementação

Estrutura de testes:

```
test/  
  screens/  
    login_screen_test.dart  
    register_screen_test.dart  
    home_screen_test.dart  
    create_plan_screen_test.dart  
  
  services/  
    api_service_test.dart  
    ai_service_test.dart  
  
  providers/  
    app_state_test.dart
```

```
integration_test/  
  auth_flow_test.dart  
  plan_flow_test.dart  
  meal_flow_test.dart  
  navigation_flow_test.dart
```

7. Controle de Testes

O acompanhamento será realizado através de:

- Casos planejados
- Casos executados
- Casos aprovados
- Casos reprovados
- Defeitos encontrados
- Evidências registradas
- Histórico de correções

8. Execução

Comandos utilizados:

```
flutter test  
flutter test integration_test
```

Execução prevista:

- Execução local
- Execução contínua via CI/CD
- Registro de logs
- Captura de evidências

9. Relatórios

Os relatórios deverão conter:

- Quantidade de testes executados
- Quantidade de testes aprovados

- Quantidade de falhas
- Evidências de execução
- Problemas encontrados
- Correções aplicadas

10. Conclusão

O processo de testes será encerrado após validação completa dos fluxos críticos do DietHub e análise dos resultados obtidos.